Karlsruhe Institute of Technology (KIT)

Modeling and Analysis in Mobility Software Engineering
Institute for Information Security and Reliability (KASTEL)
Jun.-Prof. Dr. Maike Schwammberger
76131 Karlsruhe

PRAXIS DER SOFTWAREENTWICKLUNG

UMLSL Traffic Editor

Winter Semester 2025/2026

Documentation of Code

Supervisors: Qais Hamarneh, Dr. Akhila Bairy

Project Participants:

Name	**Email Address**
Florian Portscher	uymot@student.kit.edu
Joshua Kochhan	uinfy@student.kit.edu
Mattis Bihler	ufmuk@student.kit.edu
Tao Schmidt	uqgpk@student.kit.edu
Tim Kreuch	uvvuc@student.kit.edu

Karlsruhe, Febuary 8, 2026

Table of Contents

1 Introduction

1.1 Documentation Purpose

This documentation documents the code of the program. It does not explain every file, every method or every line in detail. It explains the most important modules and concepts and gives an overview on how to extend them. Many files, such as error objects, basic utility methods and more are not documented at all or documented really short, since they are not required to understand the main code structure of the program. They have easy to understand names and comments inside the code which makes them easy to understand for anybody reading the code without much additional documentation.

1.2 Documentation Structure

The majority of the documentation focusses on explaining how the data processing in the backend works and how the models in the backend are built. Since this is organized into encapsuled modules, the documentation is also focussed on explaining the important modules on their own and give additional information on how they interact with the rest of the program.

Then there are also the view and the query evaluation. Those parts of the program are basically completely separate inside the code and don not have as strict of a module separation themselves. That´s why instead of breaking them up they are documented more as a whole unit which includes the flow of the functionality. This is easier to understand then a broken up structure.

The last part of the documentation is not focussed on the code itself. It contains a chapter with changes from the design phase, as well as a chapter with a quick summary of how to do some basic modifications. This is so that new developers don´t have to read a bunch of explanation from each module every time they want to implement new functionality, but can rather read those quick explanations.

2 Data Processing

2.1 Module: traffic_snapshot_model.py

The main module responsible for storing and processing all data from the traffic snapshot is the traffic_snapshot_model.py file. It contains the TrafficSnapshotModel class which has utility methods and manages the addition, editing and removal of cars and roads. It is also responsible for calculating the segments for the umlsl logic and handles all the segment logic.

Location: src/model/domain_models

2.1.1 Important Data Structures

- `_horizontal_roads`, `_vertical_roads`: Store roads separately which makes the segment calculation and handling easier.
- `_cars`: Stores the cars.
- `_segments`: Stores all calculated Segments (LaneSegment & CrossingSegment) indexed by uid.
- `_segments_by_lane`: Maps `Lane → [SegmentUIDs]` for easier position dependency calculation.
- `_graph` (networkx.DiGraph): Graph for modelling the neighboring segments.

2.1.2 Important Methods

recalculate_static_segments(...)

Recalculates all static segments and their connections based on the current traffic snapshot. This contains:

- Creating all `CrossingSegment` Objects at crossings.
- Creating von `LaneSegment` Objects between crossings.
- Creating and calculating the `_graph`.
- Creating the `_segments_by_lane` map.

This method is automatically called if a road is added, edited or removed.

get_valid_turn_intent_lanes(...)

Calculates valid target lanes for a cars turn intent at a crossing based on:

- Current lane
- Position
- Driving direction
- Turn intent direction

get_segment_from_lane_position(...)

Calculates a fitting segment for a given lane and position.

add_road(...) / remove_road(...) / update_road(...)

Adds, removes or updates and edited road and automatically triggers the recalculation of the segments.

add_car(...) / remove_car(...) / update_car_with_params(...)

Adds, removes or updates and edited car.

to_dict(...) / to_json(...)

Serializes and saves the current traffic snapshot data as json.

from_dict(...) / from_json(...)

Desirealizes saved traffic snapshot data and reconstructs all traffic objects.

2.1.3 Events

The model triggers corresponding events for changes:

- `CAR_ADDED`, `CAR_REMOVED`, `CAR_UPDATED`
- `ROAD_ADDED`, `ROAD_REMOVED`, `ROAD_UPDATED`

Those events are picked up by the data_controller and help isolate the logic from the user interface.

2.1.4 Interfaces

This model is never accessed directly to enforce systematic separation for read and write access from different outside modules of the program. It therefore has to interfaces: The traffic_snapshot_writer.py module, which implements all the data modulation functionality and the traffic_snapshot_reader.py module, which implements all the data reading functionality.

2.1.5 Future Extension and Modification

Adding new traffic snapshot data modifying functionality can be done by adding a new method which modifies the data structures. This method then must be implemented in the corresponding interface which can then be called from outside the module. It is important that anything which changes the road structure has to call the `_recalculate_static_segments()` method to recalculate the segments after the modification.

2.2 Module: settings_model.py

The module which is responsible for managing the settings relevant for the backend logic is the settings_model.py file. It contains the SettingsModel class which has methods for changing the settings.

Location: src/model/domain_models

2.2.1 Important Data Structures

- `breaking_deceleration`: Stores the breaking deceleration for the cars.
- `max_velocity`: Stores max velocity for the cars.

2.2.2 Important Methods

set_breaking_deceleration(...)

Changes the breaking deceleration and calls an event to notify the event controller of the change.

set_max_velocity(...)

Changes the max velocity and calls an event to notify the event controller of the change.

2.2.3 Events

The model triggers corresponding events for changes:

`CHANGE_BREAKING_DECELERATION`, `CHANGE_MAX_VELOCITY`

2.2.4 Interfaces

The methods for changing the settings can be called from outside modules, there are no additional interfaces.

2.2.5 Future Extension and Modification

Adding new settings for the backend can be done by simply adding a new attribute variable and creating a new setting method as interface.

2.3 Module: traffic_snapshot_validator.py

The module which is responsible for validating inputs and properties is the traffic_snapshot_validator.py file. It contains the TrafficSnapshotValidator class with methods which are called when for example creating cars or roads, which check the given parameters and throw an error in case of invalid arguments.

Location: src/model/domain_models

2.3.1 Important Data Structures

This module has no core data structures since it is only serves for validation which can be achieved without any internal data structures.

2.3.2 Important Methods

validate_car_params(...)

Validates car parameters from a car and throws according errors if they are invalid.

validate_car_and_autocorrect(...)

Validates a Car instance within the context of the traffic snapshot and auto corrects if possible.

...

There are many more validation methods, but they are really short and similar and therefore unnecessary to explain in detail. Those methods however have self explanatory names and comments inside the code.

2.3.3 Events

This module does not have any traditional events with outside listeners, it only has methods which throw errors. Those errors are caught and used to display error messages in the frontend.

2.3.4 Interfaces

All the validation Methods can be called from outside exactly when a corresponding validation is needed. The caller can catch the thrown validation errors and use them to deal with the validation result.

2.3.5 Future Extension and Modification

Adding new validations can simply be done by creating new methods which check the parameters to be checked and throw the necessary errors analog to the existing methods. For completely new functionality it might be necessary to add new errors as well.

2.4 Module: umlsl_queries_model.py

The main module responsible for storing and processing all queries is the umlsl_queries_model.py file. It contains the UMLSLQueriesModel class which has utility methods and manages the addition, editing and removal of queries. This model is basically a counterpart of the traffic_snapshot_model.py for the umlsl queries.

Location: src/model/domain_models

2.4.1 Important Data Structures

- `_queries`:

Stores all queries indexed by UID.

2.4.2 Important Methods

get_query_by_id(...)

Returns the query with the given UID or raises an exception if it does not exist.

get_queries(...)

Returns all queries as a plain dictionary.

add_umlsl_query(...)

Adds a new UMLSL query to the model and triggers an add event.

remove_umlsl_query(...)

Removes a query by UID and triggers a remove event.

update_umlsl_query(...)

Updates an existing query using new parameters and triggers an update event.

to_dict(...) / to_json(...)

Serializes all queries into a list of dictionaries or a JSON string.

from_dict(...) / from_json(...)

Deserializes queries from stored data and reconstructs query objects.

clear(...)

Removes all stored queries.

2.4.3 Events

The model triggers corresponding events for changes:

`UMLSL_QUERY_ADDED`, `UMLSL_QUERY_REMOVED`, `UMLSL_QUERY_UPDATED`

2.4.4 Interfaces

All the important methods can be called from outside and serve as their own interface, there are no special additional interface classes.

2.4.5 Future Extension and Modification

The data structure is an observable dictionary which automatically triggers events when manipulated. Any functionality changes for the queries themselves should be done in the query class. Adding new modifying functionality can be done by adding a new method which manipulates the dictionary in the desired way.

3 Entities

Entities are cars, roads and queries. They all extend from the Entity class. Since the entity class only holds a uid and all the three subclasses are vastly different from each other, they are each documented on their own.

3.1 Module: car.py

The car object is represented by the car.py file.

Location: src/model/entities

3.1.1 Classes

CarParams

Parameter object for creating and updating `Car` instances. Encapsulates all required and optional attributes in a separate structure.

Car (inherits from `Entity`)

Represents a car in the simulation. Validates its own state and manages its dynamic environment context.

3.1.2 Important Data Structures

- `CarParams`:

  Contains all attributes required to instantiate or update a car, including lane assignment, position, speed, and turn intent.

- `Car.environment`:

  Represents the car's local surroundings, used for movement and decision-making.

3.1.3 Important Methods

from_params(...)

Factory method that creates a new `Car` instance from a `CarParams` object and initializes its environment using the traffic snapshot reader.

validate(...)

Ensures all car attributes are valid and throws an error on failure.

update_from_params(...)

Updates an existing car's attributes from new parameters and recalculates its environment.

3.1.4 Future Extension and Modification

Changes to the car parameters must also be added to `CarParams`, `validate()` and `update_from_params()`. Changes to the turning behavior must be added in environment creation logic and validation logic for `next_turn`.

3.2 Module: road.py

The road object is represented by the road.py file.

Location: src/model/entities

3.2.1 Classes

RoadOrientation(Enum)

Represents the orientation of a road in the coordinate system.

RoadParams

Parameter object for creating and updating `Road` instances. Encapsulates all required and optional attributes in a separate structure.

Road (inherits Entity)

Represents a road as an infinite linear structure with directional lanes. Roads define geometry and lane counts but not vehicle movement.

3.2.2 Important Data Structures

RoadParams

- `name: str`
- `orientation: RoadOrientation`
- `position: float`
- `number_of_forward_lanes: int`
- `number_of_backward_lanes: int`

Road

- `name: str`
- `orientation: RoadOrientation`
- `position: float` - Position in finite axis
- `number_of_forward_lanes: int`
- `number_of_backward_lanes: int`
- `forward_lanes: list[Lane]`
- `backward_lanes: list[Lane]`

3.2.3 Important Methods

from_params(...)

Creates a `Road` instance and initializes its lanes based on the lane counts.

update_from_params(...)

Updates road attributes and adjusts lane lists accordingly. Revalidates after update.

get_bounds(...)

Returns the lower and upper spatial bounds of the road based on its orientation and lane count.

validate(...)

Ensures all attributes are valid and throws an error on failure.

3.2.4 Future Extension and Modification

Changes to the road parameters must also be added to `RoadParams`, `validate()` and `update_from_params()`.

3.3 Module: umlsl_query.py

This module contains the UMLSLQuery class, which represents a query. It also contains the UMLSLQueryParams class, which contains the parameters for creating a query.

Location: src/model/entities

3.3.1 Important Methods

update_from_params(...)

Updates query attributes.

from_params(...)

Creates a `UMLSLQuery` instance.

validate(...)

Ensures all attributes are valid and throws an error on failure.

3.3.2 Future Extension and Modification

Changes to the query parameters must also be added to `UMLSLQueryParams`, `validate()` and `update_from_params()`.

4 Commands

The whole user interaction is based on the command pattern. All commands are implemented on the same way, which is why here only the base command is documented.

4.1 Module: command.py

This module contains the Command class as well as a corresponding CommandValidationError class. The Command class is the elder of all commands.

Location: src/commands

4.1.1 Important Methods

The command only contains an abstract `execute()` method which is called when the command is executed.

4.1.2 Interfaces

The `execute()` method can be called from anywhere outside with the command reference.

4.1.3 Future Extension and Modification

To add general behavior to all commands, additional (abstract) methods can be added. To add a new command, a new subclass which contains the commands execution logic, as well a hook method in the command_controller.py and the corresponding data model has to be added.

5 Controllers

5.1 Module: application_controller.py

The application_controller.py module contains the ApplicationController class, which is the main controller that delegates all the action responsibilities to the other controllers.

Location: src/controllers

5.1.1 Important Data Structures

The application controller contains an instance of each controller, which each control a part of the programs information flow.

5.1.2 Important Methods

replace_snapshot(...)

Replaces the complete traffic snapshot and reloads triggers all important reloading/recalculating events for the replacement.

5.1.3 Interfaces

All methods can be called from outside and are designed to be called for program wide changes. The constructor of the application controller serves as an entry point for the program and is called in the main.py file.

5.1.4 Future Extension and Modification

Any new controller added to the program must be registered and set up here.

5.2 Module: event_controller.py

The event_controller.py module contains the EventController class which is responsible for handling all the events and connecting them to the frontend.

Location: src/controllers

5.2.1 Important Methods

_setup_event_listeners(...)

Connects TrafficSnapshot events to TrafficView methods using Observable pattern.

replace_models(...)

Swaps models and rewires event listeners. This is used for loading a new traffic snapshot.

_on_traffic_snapshot_event(...)

Handles events from the traffic snapshot module.

_on_settings_event(...)

Handles events from the settings module.

_on_umlsl_query_event(...)

Handles events from the query module.

5.2.2 Events

The event controller handles all event calls from the backend in different sub routines/methods and wires them to the corresponding frontend calls.

5.2.3 Interfaces

The listeners for the events serve as interfaces for the event controller since they call the corresponding methods inside the controller as soon as they detect an event.

5.2.4 Future Extension and Modification

Any new events have to be registered inside the event controller. New backend modules have to have corresponding handling methods inside the event controller.

5.3 Module: command_controller.py

The command controller controls the command pattern for the user interaction. It connects the user user inputs with corresponding infrastructure in the backend.

Location: src/controllers

5.3.1 Important Methods

_execute_command(...)

Executes a created command.

...

The command controller features an interface method for every command in existence. Each method creates a command object from the given parameters and executes it with the `_execute_command()` method.

5.3.2 Interfaces

Every of the specific command methods is an interface for the frontend. The methods can get called from outside to execute a command after a certain user interaction.

5.3.3 Future Extension and Modification

Any new command needs a new interface method inside the command controller.

5.4 Module: observables.py

The observables.py module contains classes for observable objects from the observer pattern.

Location: src/model/helper

5.4.1 Classes

Observable

This is the base class for observable objects.

ObservableDict

A dictionary that triggers observers when manipulated.

ObservableList

A list that triggers observers when manipulated.

ReadOnlyMergedDictView

An observable object which maps multiple observable dictionaries into one object for easier reading access.

5.4.2 Important Methods

All observables contain three important methods.

attach(...)

Attaches an observer callback.

detach(...)

Detaches an observer callback.

notify(...)

Notifies all observers of a manipulation.

5.4.3 Events

The classes contain methods, which notify observers when they are manipulated.

6 User Interface/View

6.1 UI Architecture

6.1.1 Qt Designer Workflow

All dialogs and windows are designed using **Qt Designer**. Each UI file contains clearly labeled:

- Text fields
- Buttons
- Spin boxes
- Dropdowns

These elements use recognizable object names to allow clean access from Python code.

The `.ui` files are stored in:

`pse/umlsl_editor/src/view/widgets/qt_widgets`

Compilation is handled by:

`pse/umlsl_editor/src/view/widgets/compile_ui.py`

This script checks whether any `.ui` files have changed and, if so, compiles them into corresponding `.py` files that define the QWidget classes.

By inheriting from both `QMainWindow` and the compiled UI class, all labeled widgets become accessible via `self.widget_name`. For example, text fields can be read and written using `self.textField.setText(...)` or `self.textField.text()`.

6.1.2 Resource Files

All `.svg` icons are stored in a Qt resource file, which is also compiled into a Python module by the same `compile_ui.py` script. This allows icons to be referenced directly in code without managing file paths manually.

6.2 List Rendering with QML

Complex lists (e.g., rows containing buttons, icons, and dynamic content) are manually designed using QML because Qt Designer does not support these layouts efficiently.

QML is used because it provides:

- **Declarative UI**: Layouts and visual structure are described directly using `ListView`, delegates, and layout components.
- **Reusable row components**: `ListRowDelegate.qml` defines a common row structure (selection behavior, edit buttons, colors), while specific lists (cars, roads, queries) inject their own content. This avoids duplication and ensures consistent styling.
- **Dynamic UI capabilities**: Conditional rendering (e.g., LaTeX image vs. fallback text), icon rotation, and hover/selection states are easier and more maintainable in QML.
- **Separation of concerns**: Python manages data and application logic, while QML focuses exclusively on presentation and interaction.

6.3 UI Updates and Signal Handling

UI updates are driven by Qt signals. These signals emit state changes or values, which the UI connects to and reacts accordingly.

All signal handling logic is implemented in:

`pse/umlsl_editor/src/view/view_event_handler_impl.py`

This ensures a centralized and consistent handling of UI state updates.

6.4 Backend Interaction

The UI communicates changes to the backend using the **Command Controller**.

The **Application Controller** is passed to most UI classes, giving them access to:

- The Command Controller
- Shared application state
- Other relevant services

This architecture ensures loose coupling between UI and backend logic.

6.5 Traffic Visualization Canvas

The traffic visualization consists of:

- A **QGraphicsView**, which handles:
  - Panning
  - Zooming
  - Coordinate system
  - Grid background
- A **QGraphicsScene**, which:
  - Holds all `QGraphicsItem` objects
  - Renders them efficiently
  - Places them within the scene's coordinate system

The main visual items include:

- `CarItem`
- `RoadItem`

Both inherit from `SelectableGraphicsItem`, which itself inherits from `QGraphicsItem`. This provides:

- Moveability
- Selectability
- Hover effects

Crossing segment items are automatically placed at road intersections and use the already explained observer pattern (the events in the backend) to listen for changes in road and car positions, updating themselves accordingly.

6.6 Keyboard Shortcuts

Keyboard shortcuts are implemented using `QShortcut` objects.

7 Query Parsing

7.1 Pipeline Overview

The overall processing pipeline is structured as follows:

Input → Lexer → ASTParser → LaTeX generation or evaluation

After parsing, the root node of the AST provides two main capabilities:

- LaTeX generation via `root.to_latex()`, which converts the parsed expression into valid LaTeX code.
- Semantic evaluation via `root.evaluate(...)`, which computes the truth value of the expression (`true` or `false`) under a given traffic state and variable assignment.

The public API for these operations is exposed by the `UMLSLEvaluator` class.

Within `evaluate_query`, the system evaluates the query against **all available views** and returns `true` if **any** view evaluates the query to `true`. This behavior is intentional and part of the semantic definition of queries.

7.2 View

A **view** represents the context in which a query is evaluated. Each view defines:

- A set of `virtual_lanes` on which the query is applied.
- A spatial interval over which the query is considered valid.
- A designated ego car.

Together, these elements determine the concrete traffic snapshot and spatial context used during evaluation.

7.3 ASTParser

The `ASTParser` is implemented as a **recursive descent parser** operating on the token stream produced by the lexer.

7.3.1 Scope and Variable Handling

The core method `parse_ast_rec` receives:

- A `start` and `end` token index defining the current parsing range.
- A list of `declared_variables`, representing exactly those variables that are in scope **between `start` and `end`**.

Quantifiers such as `exists` and `forall` introduce new variables. When such a quantifier is encountered, it extends the `declared_variables` list and then recursively calls `parse_ast_rec` on the inner expression. Because of this, parsing must begin with an empty variable list to ensure correct scoping behavior.

7.3.2 Operator Precedence

Due to the presence of infix operators, `parse_ast_rec` must determine the operator with the **lowest precedence (weakest binding)** in the current range and split the expression at that operator first. This ensures that operators are evaluated in the correct order according to the grammar.

7.3.3 Parsing Components

The parser is structured around several specialized methods:

- `parse_prefix`: Parses expressions where the operator appears first, such as `\hchop{...}{...}`. This method identifies the operator and dispatches to the corresponding handler.
- `parse_infix`: Parses expressions where the operator appears between two subexpressions, such as `true \and true`.
- `parse_atom_node`: Parses atomic expressions, such as boolean literals (`true`, `false`) or constant symbols like `\cs`.

- `parse_unary_node`: Parses unary expressions, for example `\claim{CAR}`, which take exactly one argument.
- `parse_binary_node`: Parses binary expressions, for example `\hchop{...}{...}`, which take exactly two arguments.

7.3.4 Helper Methods

Several helper methods support the core parsing logic:

- `validate_variable_name`: Checks whether a variable name is syntactically valid and not reserved.
- `find_closing_index`: Finds the token index that closes the current expression. It works by tracking the depth (balance) of opening and closing tokens and selecting the closing token where the balance returns to zero.

  This logic is implemented both for:

  - Curly braces `{ ... }`
  - Angle brackets `< ... >`
- `parse_expression_argument`: Parses arguments that themselves are full expressions, such as the argument in `\neg{expr}`.
- `parse_car_argument`: Parses arguments that are expected to denote a car.
- `parse_car`: Attempts to interpret the token at a given index as a car reference. It checks:
  - All defined cars in the current traffic snapshot.
  - All variables currently in scope.

7.4 Car Resolution

Car references in the AST are handled using the `CarResolve` abstraction, which allows nodes to defer the resolution of a car until evaluation time.

For example, in the expression `\re{c}`, the argument `c` must resolve to a car. The identifier `c` may either:

- Refer to a concrete car explicitly defined in the traffic snapshot, or
- Refer to a variable introduced by a quantifier in the current scope.

Based on this distinction, the parser creates one of two resolver types:

- `ConstantCarResolve`: Stores a direct reference to a specific car object.
- `VariableCarResolve`: Stores only the variable name and resolves it later using the evaluation context.

To obtain the actual car object, the resolver is called as follows:

```
car_resolve.resolve(variable_car_map)
```

Here, `variable_car_map` maps variable names to concrete car objects. In the case of `ConstantCarResolve`, this map is ignored, since the car reference is already fixed.

For concrete examples of how this mechanism is used, refer to the quantifier-related nodes in `quantor_nodes`.

7.5 Error Handling

Internally, `UMLSLEvaluator` always invokes the `_parse_ast` method to perform parsing.

The `ASTParser` operates purely on tokens, so the only error information it can produce consists of two token indices (`start` and `end`) indicating the range in which a syntax error occurred.

In `_parse_ast`, such errors are caught and wrapped into a `ParserError` object. The key enhancement provided by `ParserError` is the conversion of token indices into character indices:

- `startCharacterIndex`
- `endCharacterIndex`

This conversion is possible because each token stores its starting and ending character positions in the original input string. For example, the token `true` might start at character index 0 and end at character index 4. While the ASTParser itself only works with token indices, `ParserError` exposes character-level positions, which are more useful for user-facing error reporting.

8 Design Phase Changes

There have been multiple changes from the dsign phase. Their implementation is already part of the previous parts in the documentation, since they are obviously a part of the code.

8.1 Multithreading and caching

To improve the performance of the real time latex rendering, the live latex calculation and rendering is multithreaded. The generated latex images are also cached.

8.2 Precise Error Handling for UMLSL Inputs

Instead of just marking invalid queries in red, there is now a big field which displays more detailed errors for the queries. The error messages contain more detailed explanation on what part of the query is wrong.

8.3 Keyboard Shortcuts

Many of the navigation controls (saving for example) now have shortcuts to allow the user to navigate more efficiently.

9 Modification Overview

9.1 Adding new User Interaction

Since the user interaction is done with the command pattern, adding new user interaction means creating a new command. To add a new command, a corresponding command class with the execution logic has to be added. A method which creates and executes the command that can be called from the frontend has to be added in the command_controller.py. The responsible data changing model (for example the traffic snapshot model with its interface) has to get a

new method which manipulates the data and can be called from the execution method of the new command.

9.2 Adding new global Settings

To add a setting which changes the backend logic, a new setting value as well as a method which modify that value have to be added to the settings_model.py. Settings which only influence the user interface are not stored in the backend. They can be found in the view_event_handler_impl.py.